Virtual Columns from Coefficient-Position Symmetries of Binary-Polynomial Cells

As deployed in the Zinc+ reference codebase

Abstract

We describe a virtual-column construction in the Zinc+ multilinear PIOP that allows constraints to reference bit-wise rotations and right-shifts of binary-polynomial trace columns at zero marginal cost: no fresh commitment, no additional protocol round, no extra challenge, and only a single field element of communication per declared virtual column. The construction rests on a structural commutation lemma: the multilinear extension functor over the row coordinate commutes with any $\mathbb{F}_2$-linear endomorphism of the cell ring acting on the coefficient coordinate. This commutation is exploited as a *verifier-side reconstruction*: at the final challenge point, the verifier reproduces the value of the virtual MLE locally from the opened lifted evaluation of its source column. We formalise the construction, sketch its soundness, exhibit its communication-cost profile against the naive commit-and-equate approach, and observe that the same template lifts verbatim to any $\mathbb{F}_2$-linear function of the 32 coefficient bits of a cell.

Contents

1 Introduction **2**

2 Setting and Notation **3**

3 Bit-Ops and the Equivariance Lemma **4**
- 3.1 Bit-ops . . . 4
- 3.2 The equivariance lemma . . . 4
- 3.3 Equivariance under projection . . . 5
- 3.4 Commutation with row-shifts . . . 5

4 The Construction **6**
- 4.1 What is committed and what is not . . . 6
- 4.2 Constraint formulation . . . 6
- 4.3 Constraint-aggregation sumcheck . . . 7
- 4.4 Multi-point evaluation sumcheck . . . 7
- 4.5 Lifted-evaluation final check . . . 8
- 4.6 Composition with row-shifts . . . 8
- 4.7 Summary of the construction . . . 10

5 Soundness **11**
- 5.1 Threat model . . . 11
- 5.2 Failure modes . . . 11
- 5.3 Why no fresh extractor is required . . . 11
- 5.4 Quantitative summary . . . 12

6 Communication and Computation Cost **12**
6.1 Per-spec accounting . . . 12
6.2 Verifier-side computation . . . 12
6.3 Prover-side computation . . . 13
6.4 Asymptotic comparison . . . 13
6.5 Concrete numbers in the implementation . . . 13

7 Generalisation to $\mathbb{F}_2$-Linear Cell Operators **13**
7.1 The $\mathbb{F}_2$-linear envelope . . . 13
7.2 Linearity of the construction . . . 14
7.3 Beyond $\mathbb{F}_2$-linearity . . . 14

1 Introduction

A common pattern in zk-friendly arithmetizations of bit-oriented computations such as SHA-256 is the appearance, inside a single constraint, of bit-wise rotations and right-shifts of one of the trace columns. Two examples from SHA-256:

$$\sigma_0(x) = \mathrm{Rot}_7(x) \oplus \mathrm{Rot}_{18}(x) \oplus \mathrm{Sh}^{\mathrm{R}}{}_3(x),$$
$$\sigma_1(x) = \mathrm{Rot}_{17}(x) \oplus \mathrm{Rot}_{19}(x) \oplus \mathrm{Sh}^{\mathrm{R}}{}_{10}(x).$$

The naive arithmetization. Treat each rotated/shifted operand as a fresh trace column $w := \sigma_*(v)$, commit to it alongside v, and prove $w_i = \sigma(v_i)$ row-wise via additional constraints. This costs:

- one extra committed column per occurrence of σ;
- one extra equality constraint per occurrence;
- the corresponding extra opening, MLE evaluation, and proof bytes in every step downstream.

For SHA-256, where σ_0 and σ_1 are evaluated on every message-schedule row and contribute six rotation/shift instances per compression step, this overhead dominates the per-row witness budget and obscures the algebraic structure of the underlying computation.

Our observation. A binary-polynomial cell is an element of $R := \mathbb{F}_2[X]/(X^{32})$. The bit-wise operations Rot_c and $\mathrm{Sh}^{\mathrm{R}}{}_c$ act on cells as $\mathbb{F}_2$-linear endomorphisms, and they act *only on coefficient positions*: they do not depend on the row index. The multilinear extension that lifts a column $v \in R^n$ to a polynomial $\widetilde{v}$ over the row hypercube is $\mathbb{F}_2$-linear in the cell values. Because the cell coordinate and the row coordinate are independent, σ and the MLE-evaluation operator commute:

$$\widetilde{\sigma_* v}(r) \;=\; \sigma(\widetilde{v}(r)) \qquad \text{for all } r \in \mathbb{F}_q^{\nu}.$$

Implication. A virtual column $w := \sigma_* v$ does not need a commitment, an opening, or a fresh challenge. Its value at the final evaluation point of the protocol is determined by the source column's lifted opening, which the verifier already receives. The prover sends a single field element per occurrence (the value at the constraint-aggregation challenge r^*) so that the constraint polynomial can be evaluated at r^*; soundness of that single field element is bound to the source column through the existing multi-point evaluation argument and the closed-form reconstruction at the final point.

The construction is described in Sections 2 to 4, its soundness sketched in Section 5, its communication cost compared to the naive arithmetization in Section 6, and its $\mathbb{F}_2$-linear generalisation in Section 7.

2 Setting and Notation

Field, cell ring, and trace. Fix a prime q with $q \gg 2^{32}$ and write $\mathbb{F}_q$ for the field of size q. Fix a row count $n = 2^\nu$. The *cell ring* is

$$R \;:=\; \mathbb{F}_2[X]/(X^{32}),$$

a free $\mathbb{F}_2$-module of rank 32 and an $\mathbb{F}_2$-algebra. Its $\mathbb{F}_q$-lift is

$$R_q \;:=\; R \otimes_{\mathbb{F}_2} \mathbb{F}_q \;\cong\; \mathbb{F}_q[X]/(X^{32}),$$

again a free $\mathbb{F}_q$-module of rank 32. A *trace column* is an element $v \in R^n$, indexed by row $i \in \{0,1\}^\nu$.

Multilinear extension. For $v \in R^n$, the *multilinear extension* of v is the unique polynomial $\widetilde{v} \in R_q[X_1, \ldots, X_\nu]$ that is multilinear in $(X_1, \ldots, X_\nu)$ and agrees with v on $\{0,1\}^\nu$:

$$\widetilde{v}(r) \;=\; \sum_{i \in \{0,1\}^\nu} \mathrm{eq}_i(r) \cdot v_i, \qquad \mathrm{eq}_i(r) := \prod_{k=1}^{\nu} (i_k r_k + (1 - i_k)(1 - r_k)).$$

The coefficients $\mathrm{eq}_i(r) \in \mathbb{F}_q$ depend only on r and the row index i; the cell value $v_i \in R$ is $\mathbb{F}_2$-embedded in R_q. We use the same notation $\widetilde{v}(r)$ for the value at a challenge $r \in \mathbb{F}_q^\nu$, which is an element of R_q.

ψ_α-projection. Fix a transcript-bound element $\alpha \in \mathbb{F}_q$. Define the $\mathbb{F}_q$-linear projection

$$\psi_\alpha : R_q \;\longrightarrow\; \mathbb{F}_q, \qquad \psi_\alpha\Big(\textstyle\sum_{j=0}^{31} c_j X^j\Big) \;=\; \textstyle\sum_{j=0}^{31} c_j \alpha^j.$$

Composed with $\mathbb{F}_2 \hookrightarrow \mathbb{F}_q$, it descends to a map $R \to \mathbb{F}_q$ which we still denote ψ_α. The PIOP commits the multilinear extension of $\psi_\alpha(v)$, not of v itself.

Lifted evaluation. For $v \in R^n$ and $r \in \mathbb{F}_q^\nu$, define the *lifted evaluation*

$$\overline{v}(r) \;:=\; \widetilde{v}(r) \;\in\; R_q.$$

At the end of the PIOP, the prover opens $\overline{v}(r_0)$ for each base column v at a final point $r_0 \in \mathbb{F}_q^\nu$, providing all 32 $\mathbb{F}_q$-coefficients of the lifted evaluation. The verifier checks that $\psi_\alpha(\overline{v}(r_0))$ equals the value committed by the multi-point evaluation argument.

Constraints and trace rows. A UAIR specifies a constraint polynomial $C(\mathrm{up}, \mathrm{down})$ where up and down are abstract trace rows. The up row exposes the cells of trace columns at row i; the down row exposes *virtual* cells, namely:

- *Row-shift virtual columns:* $\mathrm{sh}_\delta v$ at row i is $v_{i+\delta}$. These are already part of the protocol.
- *Bit-op virtual columns:* the construction of this paper.

Index sets. Let $\mathcal{S}_{\mathrm{up}} = \{1, \ldots, K\}$ be the set of base trace column indices. Let $\mathcal{S}_{\mathrm{sh}} = \{(j, \delta_\ell)\}$ be the set of row-shift specs. We introduce a third index set

$$\mathcal{S}_{\mathrm{bit}} \;=\; \{(j_\ell, \sigma_\ell)\}_{\ell=1}^{m}$$

of *bit-op specs*, where $j_\ell \in \mathcal{S}_{\mathrm{up}}$ is a base column index and σ_ℓ is a bit-op (defined next).

3 Bit-Ops and the Equivariance Lemma

3.1 Bit-ops

Definition 3.1 (Bit-op)**.** A *bit-op* is an $\mathbb{F}_2$-linear endomorphism $\sigma \in \mathrm{End}_{\mathbb{F}_2}(R)$ that acts as a permutation

$$p_\sigma : \{0, \dots, 31\} \longrightarrow \{0, \dots, 31\} \cup \{\perp\}$$

on coefficient positions, in the sense that

$$\sigma\Big(\sum_{j=0}^{31} c_j X^j\Big) = \sum_{j\,:\,p_\sigma(j)\neq\perp} c_j\, X^{p_\sigma(j)}.$$

The two specific bit-ops considered in the implementation are, for $c \in \{0, \dots, 31\}$:

$$\begin{aligned} \mathrm{Rot}_c : &\quad p_{\mathrm{Rot}_c}(j) = (j + c) \bmod 32, \\ \mathrm{Sh^R}_c : &\quad p_{\mathrm{Sh^R}_c}(j) = j - c \text{ if } j \geq c, \text{ else } \perp. \end{aligned}$$

Remark 3.2. A bit-op extends $\mathbb{F}_q$-linearly to $R_q \to R_q$ by acting on coefficient positions in R_q exactly as on R. We denote the extension by the same symbol σ.

Definition 3.3 (Cell-wise action on columns)**.** For a bit-op σ and a column $v \in R^n$, define $\sigma_*(v) \in R^n$ by $(\sigma_*(v))_i := \sigma(v_i)$.

3.2 The equivariance lemma

Lemma 3.4 (Equivariance of MLE under bit-ops)**.** *For any bit-op $\sigma \in \mathrm{End}_{\mathbb{F}_2}(R)$, any column $v \in R^n$, and any $r \in \mathbb{F}_q^\nu$,*

$$\widetilde{\sigma_* v}(r) = \sigma(\widetilde{v}(r)) \qquad \text{in } R_q. \tag{1}$$

Equivalently, the diagram

$$\begin{array}{ccc} R^n & \xrightarrow{\sigma_*} & R^n \\ \downarrow \widetilde{(\,\cdot\,)}(r) & & \downarrow \widetilde{(\,\cdot\,)}(r) \\ R_q & \xrightarrow{\sigma} & R_q \end{array}$$

commutes.

Proof. The MLE-evaluation map at a fixed point $r \in \mathbb{F}_q^\nu$ is

$$T_r : R^n \longrightarrow R_q, \qquad T_r(v) = \sum_{i \in \{0,1\}^\nu} \mathrm{eq}_i(r) \cdot v_i,$$

with scalar coefficients $\mathrm{eq}_i(r) \in \mathbb{F}_q$. The map is $\mathbb{F}_q$-linear in the R_q-valued vector $(v_i)_i$, hence in particular $\mathbb{F}_2$-linear in the cell coordinate $v_i \in R$.

The bit-op $\sigma \in \mathrm{End}_{\mathbb{F}_2}(R)$ acts trivially on the row coordinate (it is the same map at every row i) and is $\mathbb{F}_2$-linear in the cell coordinate. Since the row coordinate and the cell coordinate of $R^n = R \otimes_{\mathbb{F}_2} \mathbb{F}_2^n$ are independent and the action of σ commutes with scalar multiplication by $\mathrm{eq}_i(r) \in \mathbb{F}_q$ (which lives in the row coordinate after extension of scalars), we have

$$\begin{aligned} T_r(\sigma_* v) &= \sum_i \mathrm{eq}_i(r) \cdot \sigma(v_i) \\ &= \sum_i \sigma\big(\mathrm{eq}_i(r) \cdot v_i\big) \qquad (\sigma \text{ is } \mathbb{F}_q\text{-linear after extension}) \\ &= \sigma\Big(\sum_i \mathrm{eq}_i(r) \cdot v_i\Big) \qquad (\sigma \text{ is additive}) \\ &= \sigma\big(T_r(v)\big). \end{aligned}$$

□

3.3 Equivariance under projection

Corollary 3.5 (Projected equivariance)**.** *For any bit-op σ, any column $v \in R^n$, any $\alpha \in \mathbb{F}_q$, and any $r \in \mathbb{F}_q^\nu$,*

$$\widetilde{\psi_\alpha(\sigma_* v)}(r) \;=\; \psi_\alpha(\sigma(\overline{v}(r))).$$

Proof. Both ψ_α and σ are $\mathbb{F}_q$-linear maps on R_q, and ψ_α is also linear in the row coordinate (it acts on the cell coordinate). By Theorem 3.4 applied to the column $\psi_\alpha(\sigma_* v) \in \mathbb{F}_q^n$, viewed cell-wise, and by linearity of ψ_α,

$$\widetilde{\psi_\alpha(\sigma_* v)}(r) \;=\; \psi_\alpha(\widetilde{\sigma_* v}(r)) \;=\; \psi_\alpha(\sigma(\widetilde{v}(r))) \;=\; \psi_\alpha(\sigma(\overline{v}(r))).$$

□

Theorem 3.5 is the operative form: it tells the verifier that the value of a virtual bit-op MLE at the final point r_0 can be reconstructed from the lifted opening of the source column by applying σ to a polynomial in R_q and then ψ_α, both of which are local field-arithmetic operations of cost $O(32)$.

Remark 3.6 (Geometric reading). Under the identification $R^n \cong R \otimes_{\mathbb{F}_2} \mathbb{F}_2^n$, σ_* acts on the left tensor factor only and the MLE evaluation T_r is induced from a map on the right tensor factor only. Theorem 3.4 is the assertion that the actions on disjoint tensor factors commute, i.e. that a tensor product of operators is well-defined.

3.4 Commutation with row-shifts

Row-shifts act on the row coordinate. Fix $\delta \in \{0, \ldots, n-1\}$ and define $\mathrm{sh}_\delta : R^n \to R^n$ by

$$(\mathrm{sh}_\delta v)_i \;=\; \begin{cases} v_{i+\delta} & \text{if } i + \delta < n, \\ 0 & \text{otherwise,} \end{cases}$$

where $i \in \{0,1\}^\nu$ is identified with its base-2 encoded integer in $\{0, \ldots, n-1\}$ and arithmetic is over $\mathbb{Z}$.

Why row-shift is not a point shift. It is tempting to seek a transformation $\mathrm{shift}_\delta : \mathbb{F}_q^\nu \to \mathbb{F}_q^\nu$ satisfying $\widetilde{\mathrm{sh}_\delta v}(r) = \widetilde{v}(\mathrm{shift}_\delta(r))$ for all v. No such map exists in general: the integer-shift permutation on $\{0, \ldots, n-1\}$ does not extend to an $\mathbb{F}_q$-affine multilinear-shift compatible map on $\mathbb{F}_q^\nu$ (the truncation at the top of the cube is the obstruction). The host PIOP expresses the row-shift identity in *kernel form* instead.

Lemma 3.7 (Row-shift kernel identity)**.** *There exists a bivariate polynomial* $\mathrm{next}_\delta(r, b)$*, multilinear in $r \in \mathbb{F}_q^\nu$ for each fixed $b \in \{0,1\}^\nu$, depending only on δ and not on v, such that for every $v \in R^n$ and every $r \in \mathbb{F}_q^\nu$,*

$$\widetilde{\mathrm{sh}_\delta v}(r) \;=\; \sum_{b \in \{0,1\}^\nu} \mathrm{next}_\delta(r, b) \cdot v_b. \tag{2}$$

A closed form on the boolean indexing $b \in \{0,1\}^\nu$ is

$$\mathrm{next}_\delta(r, b) \;=\; \begin{cases} \mathrm{eq}(r,\, b - \delta) & \textit{if } b \geq \delta, \\ 0 & \textit{if } b < \delta, \end{cases}$$

extended multilinearly in r to all of $\mathbb{F}_q^\nu$.

Proof. By definition of multilinear extension, $\widetilde{\mathrm{sh}_\delta v}(r) = \sum_{i\in\{0,1\}^\nu} \mathrm{eq}(r,i)\cdot(\mathrm{sh}_\delta v)_i$. Re-indexing $b := i+\delta$ and using $(\mathrm{sh}_\delta v)_i = 0$ for $i+\delta \geq n$,

$$\widetilde{\mathrm{sh}_\delta v}(r) \;=\; \sum_{b\geq\delta} \mathrm{eq}(r, b-\delta)\cdot v_b \;=\; \sum_{b\in\{0,1\}^\nu} \mathrm{next}_\delta(r,b)\cdot v_b. \qquad \square$$

Equation (2) expresses the row-shifted MLE as a fixed $\mathbb{F}_q$-linear functional of the base column v, with weights $\mathrm{next}_\delta(r,b)$ that depend on δ and r but not on v. The host PIOP's multi-point evaluation argument is built around this kernel: it pairs the kernel as a selector MLE with the base column MLE inside a single sumcheck (see Section 4.4).

Corollary 3.8 (Bit-op and row-shift commute)**.** *For any bit-op σ, any row-shift δ, and any column $v \in R^n$,*

$$\sigma_*(\mathrm{sh}_\delta v) \;=\; \mathrm{sh}_\delta(\sigma_* v) \quad \textit{in } R^n, \tag{3}$$

and consequently

$$\widetilde{\sigma_*(\mathrm{sh}_\delta v)}(r) \;=\; \sum_{b\in\{0,1\}^\nu} \mathrm{next}_\delta(r,b)\cdot\sigma(v_b). \tag{4}$$

Proof. For every row i, both columns in eq. (3) take value $\sigma(v_{i+\delta})$ when $i+\delta < n$ and 0 otherwise (using $\sigma(0) = 0$ since σ is $\mathbb{F}_2$-linear). For eq. (4), apply Theorem 3.4 to $\mathrm{sh}_\delta v$:

$$\widetilde{\sigma_*(\mathrm{sh}_\delta v)}(r) \;=\; \sigma\big(\widetilde{\mathrm{sh}_\delta v}(r)\big) \;=\; \sigma\Big(\sum_b \mathrm{next}_\delta(r,b)\cdot v_b\Big),$$

then use $\mathbb{F}_q$-linearity of σ on R_q (Theorem 3.2) to push σ inside the sum. $\square$

Theorem 3.8 is the structural reason the construction of Section 4 extends to row-shifted bit-op virtual columns at no additional protocol cost: see Section 4.6.

4 The Construction

We describe how the index set $\mathcal{S}_{\mathrm{bit}}$ is folded into the multilinear PIOP. The presentation tracks the four protocol stages relevant to trace-column claims: *(i)* ideal check setup; *(ii)* constraint-aggregation sumcheck; *(iii)* multi-point evaluation sumcheck; *(iv)* lifted-evaluation final check.

4.1 What is committed and what is not

The construction does *not* alter the commitment phase. The prover commits exactly to the multilinear extensions of $\psi_\alpha(v_j)$ for $j \in \mathcal{S}_{\mathrm{up}}$. No bit-op virtual column is committed. The set $\mathcal{S}_{\mathrm{bit}}$ is fixed by the UAIR signature and is part of the verifier's public parameters; it is not chosen interactively.

4.2 Constraint formulation

A UAIR with $|\mathcal{S}_{\mathrm{bit}}| = m$ admits constraint polynomials of the form

$$C(\,\mathrm{up},\ \mathrm{down}_{\mathrm{sh}},\ \mathrm{down}_{\mathrm{bit}}\,) \;=\; 0$$

where the $\mathrm{down}_{\mathrm{bit}}$ slots correspond to virtual cells $\sigma_\ell(v_{j_\ell})$ for $\ell = 1,\ldots,m$, in the order fixed by the signature.

4.3 Constraint-aggregation sumcheck

The multilinear PIOP runs, at random point $r^* \in \mathbb{F}_q^\nu$, a sumcheck on the constraint-aggregation polynomial. The prover materialises, in addition to the existing MLE inputs (last-row selector, eq_{r^*}, base trace MLEs, row-shift MLEs), the following *virtual MLEs*:

$$M_\ell(\,\cdot\,) \;:=\; \widetilde{\psi_\alpha(\sigma_{\ell\,*} v_{j_\ell})}, \qquad \ell = 1, \ldots, m.$$

By Theorem 3.5, M_ℓ is the multilinear extension of the column $\psi_\alpha \circ \sigma_\ell$ applied cell-wise to v_{j_ℓ}. Operationally, M_ℓ is computed in $O(n)$ cell evaluations: for each row i, the bit-op acts on the 32 coefficients of $v_{j_\ell,i}$ as a permutation, and the resulting cell is projected by ψ_α to a scalar in $\mathbb{F}_q$.

The sumcheck is run on the polynomial

$$\mathrm{Sel}(r^*) \cdot \mathrm{eq}_{r^*} \cdot C(\mathrm{up}_{\mathrm{eval}},\ \mathrm{down}_{\mathrm{sh,eval}},\ \mathrm{down}_{\mathrm{bit,eval}})$$

where $\mathrm{down}_{\mathrm{bit,eval}}$ at row i is read from M_ℓ. At the end of the sumcheck, after ν rounds, the verifier obtains:

- base evaluations $\big(\widetilde{\psi_\alpha(v_j)}(r^{**})\big)_{j \in \mathcal{S}_{\mathrm{up}}}$,
- row-shift evaluations $\big(\widetilde{\psi_\alpha(\mathrm{sh}_\delta\, v_j)}(r^{**})\big)_{(j,\delta) \in \mathcal{S}_{\mathrm{sh}}}$,
- bit-op evaluations $\big(b_\ell^{**} := M_\ell(r^{**})\big)_{\ell=1,\ldots,m}$,

where $r^{**} \in \mathbb{F}_q^\nu$ is the sumcheck challenge.[1]

The vector $(b_1^{**}, \ldots, b_m^{**})$ is sent by the prover and absorbed into the transcript, contributing exactly m field elements to the proof size. It enters the verifier's claim-value reconstruction of C at the sumcheck endpoint via the $\mathrm{down}_{\mathrm{bit}}$ slots; tampering any b_ℓ^{**} violates the sumcheck claim with overwhelming probability.

4.4 Multi-point evaluation sumcheck

The multi-point evaluation sub-protocol (mp_eval) reduces a list of MLE evaluation claims at r^{**} — including row-shifted variants of those MLEs — to a *single residual claim per source MLE* at a common final point $r_0 \in \mathbb{F}_q^\nu$. The reduction uses the row-shift kernel identity eq. (2): a row-shift of v_j by δ at the point r^{**} is a fixed $\mathbb{F}_q$-linear functional of the same source's cell values, with weights given by $\mathrm{next}_\delta(r^{**}, \cdot)$. mp_eval pairs this kernel with the base source MLE inside a single sumcheck whose endpoint is r_0.

Sumcheck body. Concretely, mp_eval runs a sumcheck on

$$\sum_{b \in \{0,1\}^\nu} \Big[\, \mathrm{eq}(r^{**}, b) \cdot \sum_{j \in \mathcal{S}_{\mathrm{up}}} \gamma_j\, \widetilde{\psi_\alpha(v_j)}(b) \;+\; \sum_{(j,\delta) \in \mathcal{S}_{\mathrm{sh}}} \alpha_{(j,\delta)}\, \mathrm{next}_\delta(r^{**}, b) \cdot \widetilde{\psi_\alpha(v_j)}(b) \,\Big], \tag{5}$$

where the γ_j batch the up-claims across base sources and the $\alpha_{(j,\delta)}$ batch the row-shift down-claims; both are transcript challenges. The expected sum equals $\sum_j \gamma_j\, \mathrm{up_eval}_j + \sum_{(j,\delta)} \alpha_{(j,\delta)}\, \mathrm{down_eval}_{(j,\delta)}$, the random combination of the prover's claims at r^{**}.

The construction extends eq. (5) to include the bit-op virtual sources. Define

$$\mathcal{F} \;:=\; \underbrace{\{\, \widetilde{\psi_\alpha(v_j)} \,:\, j \in \mathcal{S}_{\mathrm{up}} \,\}}_{\text{base sources}} \cup \underbrace{\{\, M_\ell \,:\, \ell = 1, \ldots, m \,\}}_{\text{bit-op virtual sources}}.$$

The shift list $\mathcal{S}_{\mathrm{sh}}$ is unchanged: it continues to reference base sources (extending it to bit-op virtual sources is a free operation, discussed in Section 4.6). The bit-op virtual sources enter mp_eval as additional up-claims at r^{**} with values $b_1^{**}, \ldots, b_m^{**}$, contributing fresh batching coefficients γ_ℓ and adding the term $\mathrm{eq}(r^{**}, b) \cdot \gamma_\ell M_\ell(b)$ inside the bracket of eq. (5).

[1] The implementation reuses the symbol r^* for this final sumcheck point. We use r^{**} here to keep the prior r^* available if needed.

Residual structure at r_0. After ν sumcheck rounds, the prover has been pinned to a single value of the bracketed combination at $b = r_0$:

$$\mathrm{eq}(r^{**}, r_0)\sum_j \gamma_j\, \widetilde{\psi_\alpha(v_j)}(r_0) \;+\; \sum_{(j,\delta)\in\mathcal{S}_{\mathrm{sh}}} \alpha_{(j,\delta)}\, \mathrm{next}_\delta(r^{**}, r_0)\cdot \widetilde{\psi_\alpha(v_j)}(r_0) \;+\sum_\ell \mathrm{eq}(r^{**}, r_0)\, \gamma_\ell\, M_\ell(r_0).$$

The selector values $\mathrm{eq}(r^{**}, r_0)$ and $\mathrm{next}_\delta(r^{**}, r_0)$ are *scalars* the verifier computes locally from the transcript. The expression is therefore an $\mathbb{F}_q$-linear combination of one residual per source $f \in \mathcal{F}$, all evaluated at the common point r_0:

- For each base source $j \in \mathcal{S}_{\mathrm{up}}$: $e_j := \widetilde{\psi_\alpha(v_j)}(r_0)$.
- For each bit-op virtual source ℓ: $e_\ell := M_\ell(r_0)$.

No row-shift residual survives at r_0: a row-shift claim (j, δ) contributes $\alpha_{(j,\delta)} \cdot \mathrm{next}_\delta(r^{**}, r_0) \cdot e_j$, i.e. it is absorbed into e_j as a coefficient. The output of mp_eval is one residual per element of $\mathcal{F}$, indexed by sources only.

4.5 Lifted-evaluation final check

The prover opens, per base column $j \in \mathcal{S}_{\mathrm{up}}$, the lifted evaluation $\overline{v_j}(r_0) \in R_q$ (i.e., all 32 $\mathbb{F}_q$-coefficients). The verifier produces *open evals* — one per source in $\mathcal{F}$ — by:

- For each base source $j \in \mathcal{S}_{\mathrm{up}}$:
$$\tilde{e}_j \;:=\; \psi_\alpha(\overline{v_j}(r_0)). \tag{6}$$
- For each bit-op virtual source $\ell = 1, \ldots, m$, by Theorem 3.5:
$$\tilde{e}_\ell \;:=\; \psi_\alpha(\sigma_\ell(\overline{v_{j_\ell}}(r_0))). \tag{7}$$

The verifier passes $(\tilde{e}_*)$ to mp_eval's subclaim verifier, which checks a single combined identity:

$$\mathrm{Subclaim}_{\mathrm{mp_eval}}(r_0;\, (\tilde{e}_*),\, \mathcal{S}_{\mathrm{sh}},\, \mathrm{transcript}) \;\stackrel{?}{=}\; 0. \tag{8}$$

There is no separate "base check" or "row-shift check" at the final stage: Equation (8) is the unique closing identity of the protocol.

Why this discharges all three failure modes. Honest data satisfies eq. (8). Conversely, any of: *(i)* a tampered base lifted opening $\overline{v_j}(r_0)$, *(ii)* a tampered bit-op claim b_ℓ^{**} at r^{**}, or *(iii)* an inconsistent reconstruction of $\tilde{e}_\ell$ via eq. (7) — breaks eq. (8) with overwhelming probability over r^{**}, r_0, and α. The formal soundness argument is in Section 5.

4.6 Composition with row-shifts

A natural question is how bit-op virtual columns interact with the row-shift virtual columns the host PIOP already supports. There are three statements to make.

Coexistence. A UAIR may freely declare both $(j, \delta) \in \mathcal{S}_{\mathrm{sh}}$ and $(j, \sigma) \in \mathcal{S}_{\mathrm{bit}}$ on the same base column v_j. The two virtual columns

$$\mathrm{sh}_\delta v_j \quad \text{and} \quad \sigma_*(v_j)$$

are independent objects: each is materialised separately during the constraint-aggregation sumcheck, each contributes its own claim-at-r^{**}, and each enters mp_eval through its own slot — a row-shift slot for the former, an additional source for the latter. They do not interact.

Commutation. If a UAIR's constraints reference the *composition* $\sigma(v_j[t+\delta])$ — i.e., the bit-op applied to a row-shifted cell — Theorem 3.8 applies. The column-level identity

$$\sigma_*(\mathrm{sh}_\delta v_j) \;=\; \mathrm{sh}_\delta(\sigma_*\, v_j)$$

implies that the bit-op-of-row-shift and the row-shift-of-bit-op are the same column. The MLE-level identity eq. (4) expresses evaluation of the composed virtual MLE at r as the row-shift kernel applied to the bit-op-acted base column:

$$\widetilde{\sigma_*(\mathrm{sh}_\delta v_j)}(r) \;=\; \sum_{b\in\{0,1\}^\nu} \mathrm{next}_\delta(r,b)\cdot\sigma(v_{j,b}).$$

Deployed pattern: commit-and-pin. The current Zinc+ SHA-256 arithmetization needs $\sigma_0(W[t-15])$ and $\sigma_1(W[t-2])$ on the message-schedule equation. Rather than introduce a primitive for the composed virtual column, the deployed UAIR uses a *commit-and-pin* pattern:

1. Commit a fresh row-local intermediate column W_{σ_0} with $W_{\sigma_0}[t] := \sigma_0(W[t])$.
2. Declare the bit-op specs $(j_W, \mathrm{Rot}_{25}), (j_W, \mathrm{Rot}_{14}), (j_W, \mathrm{ShR}_3) \in \mathcal{S}_{\mathrm{bit}}$ on the base column W — all row-local, no row-shift.
3. Add a row-local algebraic constraint that pins $W_{\sigma_0}[t]$ to the bit-op virtuals:

$$\mathrm{Rot}_{25}(W[t]) + \mathrm{Rot}_{14}(W[t]) + \mathrm{ShR}_3(W[t]) \;-\; W_{\sigma_0}[t] \;-\; 2\cdot \mathrm{ov}_{\sigma_0}[t] \;\equiv\; 0 \pmod{X-2},$$

   where the ov_{σ_0} overflow column absorbs the $\mathrm{F}_2[X] \to \mathbb{Q}[X]$ lift carry.
4. Access $\sigma_0(W[t-15])$ in the schedule equation as a standard *row-shift of* W_{σ_0}, declared by an ordinary $(W_{\sigma_0}, \delta) \in \mathcal{S}_{\mathrm{sh}}$ — handled by the host PIOP's existing row-shift machinery, no new primitive.

The cost is one extra committed column per composed bit-op operand (plus a row-local pinning constraint and a small ov column), but the bit-op specs and the row-shift specs both stay within their existing primitive types.

Extensibility (not currently deployed). As a forward-looking design point: the construction admits, with no new soundness loss and no new round, a third virtual-column primitive

$$\mathcal{S}^{\mathrm{sh}}_{\mathrm{bit}} \;=\; \{(j_\ell, \delta_\ell, \sigma_\ell)\}_{\ell=1}^{m'}$$

of *row-shifted bit-op virtual columns*, encoding the cell $\sigma_\ell(v_{j_\ell}[t+\delta_\ell])$ as a single virtual column *with no committed intermediate*. The protocol bookkeeping for each spec $(j_\ell, \delta_\ell, \sigma_\ell)$ would be:

1. In the constraint-aggregation sumcheck, the prover materialises

$$M^{\mathrm{sh}}_\ell \;:=\; \psi_\alpha(\widetilde{\sigma_{\ell\,*}(\mathrm{sh}_{\delta_\ell} v_{j_\ell})})$$

   (cell-wise, $O(n)$ work per spec).
2. One field element $b^{\mathrm{sh},**}_\ell := M^{\mathrm{sh}}_\ell(r^{**})$ is sent at the end of the sumcheck, absorbed into the transcript, and contributes to the constraint-evaluation reconstruction at r^{**}.
3. mp_eval admits the spec equivalently as either:
   - an additional *source* (with up-claim $b^{\mathrm{sh},**}_\ell$ at r^{**}, no row-shift), or
   - a row-shift on the bit-op virtual source $M_{(j_\ell,\sigma_\ell)}$ (with down-claim $b^{\mathrm{sh},**}_\ell$ at r^{**}, shift δ_ℓ).

By eq. (4), both routes produce the same residual at r_0.

4. In either route, the row-shift contribution at r_0 is the *scalar* $\mathrm{next}_{\delta_\ell}(r^{**}, r_0)$ (computed by the verifier locally from the transcript) multiplied by a residual reconstructed from the same lifted opening $\overline{v_{j_\ell}}(r_0)$ as in the unshifted case:

$$\psi_\alpha\big(\sigma_\ell(\overline{v_{j_\ell}}(r_0))\big).$$

No additional lifted opening, no separate residual, and no shifted-point evaluation is required.

The extension would cost one additional MLE materialisation and one additional field element per declared (j, δ, σ) spec, and add no new round, no new commitment, and no new lifted opening — but it requires (i) extending `BitOpSpec` or `ShiftSpec` with a composite-spec variant, (ii) wiring the new spec into CPR's `build_bit_op_mles` and into mp_eval's source/shift lists, and (iii) extending the verifier's r_0 reconstruction. The current codebase does not implement this primitive; it uses the commit-and-pin pattern above instead.

Trade-off summary. Per composed bit-op operand $\sigma(v[t+\delta])$ in a constraint:

- *Deployed (commit-and-pin)*: +1 committed column, +1 row-local pinning constraint, +1 overflow column, plus standard row-shift of the committed intermediate.
- *Extensibility*: +1 field element in the proof, plus the prover-side MLE materialisation; 0 commitments, 0 extra constraints.

The deployed pattern was chosen for implementation simplicity and because the per-row constraint cost is amortised against the bit-op-virtual savings within the same UAIR.

4.7 Summary of the construction

Construction 4.1 (Bit-op virtual columns)**.** Given a UAIR signature with bit-op specs $\mathcal{S}_{\mathrm{bit}} = \{(j_\ell, \sigma_\ell)\}_{\ell=1}^m$:

1. No commitment is performed for any $\mathcal{S}_{\mathrm{bit}}$ entry.
2. In the constraint-aggregation sumcheck, the prover materialises $M_\ell := \widetilde{\psi_\alpha(\sigma_\ell * v_{j_\ell})}$ using the source column committed for v_{j_ℓ}, evaluated cell-wise.
3. At the end of that sumcheck, the prover sends one field element $b_\ell^{**} := M_\ell(r^{**})$ per spec, which is absorbed into the transcript and used in the claim-value reconstruction.
4. In the multi-point evaluation sumcheck, the source list is extended by $\{M_\ell\}_{\ell=1}^m$, with up-claim values $\{b_\ell^{**}\}$ at r^{**}. mp_eval reduces all claims — base up, base row-shifts, and bit-op up — to one residual per source at the common final point r_0.
5. At the final-check stage, the verifier supplies open evals eq. (6), eq. (7) to mp_eval's subclaim verifier; the protocol accepts iff the single subclaim equation eq. (8) holds.

Composed access $\sigma(v_{j_\ell}[t+\delta])$ is supported in the deployed implementation via a commit-and-pin pattern (one extra row-local committed column with a pinning constraint), and admits a free protocol-level extension via a (j, δ, σ) primitive that the current codebase does not yet implement; see Section 4.6.

5 Soundness

We argue informally that Theorem 4.1 preserves the knowledge-soundness error of the host PIOP up to a negligible additive term linear in the size of $\mathcal{S}_{\text{bit}}$. The argument is structural: every new claim added by the construction is bound to existing claims either algebraically (sumcheck claim-value reconstruction) or by closed-form substitution (Equation (7)).

5.1 Threat model

We assume the host PIOP is knowledge-sound when restricted to the existing classes of trace claims (base columns and row-shift virtual columns). Let ϵ_{host} denote its soundness error. We argue that the augmented PIOP has soundness error

$$\epsilon_{\text{host}} \;+\; m \cdot \epsilon_{\text{mp}} \;+\; \text{negl}(\lambda),$$

where ϵ_{mp} is the per-source soundness loss of the multi-point evaluation argument (typically ν/q), and $m = |\mathcal{S}_{\text{bit}}|$.

5.2 Failure modes

A malicious prover $\mathcal{P}^*$ can attempt to cheat in at most three places:

(F1) The base column's lifted opening $\overline{v_{j_\ell}}(r_0)$ is wrong. The verifier reads the lifted opening to form the open evals eq. (6) and substitutes them into the mp_eval subclaim eq. (8). If $\overline{v_{j_\ell}}(r_0) \neq \widetilde{v_{j_\ell}}(r_0)$, then with probability $\geq 1 - \text{negl}(\lambda)$ over α and r_0, $\psi_\alpha(\overline{v_{j_\ell}}(r_0)) \neq \widetilde{\psi_\alpha(v_{j_\ell})}(r_0)$, and eq. (8) fails. This is a Schwartz–Zippel argument on the polynomial that interpolates the committed projected MLE, identical to the host PIOP and unchanged by our construction.

(F2) The bit-op claim at r^{**} is wrong. Suppose b_ℓ^{**} disagrees with $M_\ell(r^{**})$ for some ℓ. Conditioned on the sumcheck claim-value equation passing, this implies the constraint polynomial C evaluates to a value at (r^*, r^{**}) that is consistent with the prover's answers but not with any honest trace; standard sumcheck soundness pins this event to probability $\leq \nu \deg(C)/q$. If instead the prover compensates by tampering $\text{down}_{\text{bit,eval}}$ in mp_eval consistently with b_ℓ^{**}, the residual claim e_ℓ at r_0 will be inconsistent with the equivariance identity, caught by (F3).

(F3) The bit-op residual claim at r_0 is wrong. Conditioned on (F1) not occurring, the verifier holds the correct $\overline{v_{j_\ell}}(r_0)$, and eq. (7) yields the correct

$$M_\ell(r_0) \;=\; \psi_\alpha\big(\sigma_\ell(\overline{v_{j_\ell}}(r_0))\big)$$

deterministically — *the prover has no freedom over $\tilde{e}_\ell$.* The mp_eval subclaim eq. (8) therefore reduces to a single algebraic identity whose left- and right-hand sides match if and only if every $b_{\ell'}^{**}$ was consistent with $M_{\ell'}(r^{**})$ (and likewise for the base sources and their row-shifts). By multi-point-evaluation soundness, an incorrect b_ℓ^{**} propagates to a violation of eq. (8) with probability at least $1 - \nu/q$ over r_0. We pay this loss once per extra source, hence $m \cdot \epsilon_{\text{mp}}$ overall.

5.3 Why no fresh extractor is required

A standard concern when adding new prover messages is whether extraction of the witness needs new machinery. Here, the only new prover messages are the m scalars $\{b_\ell^{**}\}$, and they are *not* commitments — they are values of MLEs that the verifier can recompute (in the soundness analysis, by virtue of holding the extracted base columns).

Concretely, a knowledge extractor $\mathcal{E}$ for the augmented PIOP runs the host extractor unchanged to recover the base columns $(v_j)_{j \in \mathcal{S}_{\mathrm{up}}}$. It then defines the bit-op virtual columns by $\sigma_{\ell *}(v_{j_\ell})$ — these are deterministic functions of the base columns. Soundness of the host PIOP guarantees the base columns satisfy the host's constraints. By Theorem 3.4, $\widetilde{\sigma_{\ell *} v_{j_\ell}}$ agrees with M_ℓ on the boolean cube, hence as multilinear polynomials. Therefore the augmented constraint C — which by hypothesis was satisfied at the sumcheck endpoint — is satisfied row-wise by the extracted columns. The extractor needs no new randomness, no new oracle queries, and no new rewinding beyond what the host PIOP already does.

5.4 Quantitative summary

Proposition 5.1 (Soundness, informal)**.** *Let Π denote the host multilinear PIOP and Π^+ its augmentation by Theorem 4.1 with bit-op set $\mathcal{S}_{\mathrm{bit}}$ of size m. If Π has knowledge-soundness error ϵ_{host}, then Π^+ has knowledge-soundness error*

$$\epsilon_{\mathrm{host}} \; + \; m \cdot \epsilon_{\mathrm{mp}}.$$

The $m \cdot \epsilon_{\mathrm{mp}}$ term is identical in form to the soundness loss the host PIOP already pays per row-shift virtual column.

6 Communication and Computation Cost

We compare Theorem 4.1 against the naive arithmetization that materialises a fresh trace column $w := \sigma_*(v)$, commits to it, and proves $w_i = \sigma(v_i)$ for every row i.

6.1 Per-spec accounting

Per declared spec $(j_\ell, \sigma_\ell) \in \mathcal{S}_{\mathrm{bit}}$:

	Commitment	Sumcheck rounds	Field elements (proof)	New challenges
Naive (commit-and-equate)	1 column	$\Theta(\log n)$ extra	$\Theta(\log n)$ + opening	≥ 1
This construction	0	0	1	0

Table 1: Per-spec cost. The naive count assumes the equality $w_i = \sigma(v_i)$ is proven by a separate sumcheck or batched into the existing aggregation.

6.2 Verifier-side computation

Verifier-side, per spec, the construction adds:

- One $\mathbb{F}_q$-linear permutation of 32 coefficients (σ_ℓ on R_q): $O(32)$ assignments;
- One ψ_α-projection of a degree-< 32 polynomial in R_q: 32 multiplications and 31 additions in $\mathbb{F}_q$ (or $O(32)$ multiplications if α-powers are precomputed);
- One substitution into the mp_eval subclaim equation: $O(1)$ field operations.

Total: $O(32)$ $\mathbb{F}_q$-operations per spec, independent of n.

6.3 Prover-side computation

Prover-side, per spec, the construction adds:

- Materialisation of M_ℓ: n rows, each $O(32)$ $\mathbb{F}_q$-operations to apply $\psi_\alpha \circ \sigma_\ell$;
- Inclusion of M_ℓ in the constraint-aggregation sumcheck: one extra MLE in the sumcheck working set, contributing $O(n)$ additional folding work per round (i.e., $O(n\nu)$ total over the ν rounds), comparable to one extra row-shift virtual column;
- Inclusion of M_ℓ as a source in mp_eval: one extra source claim, contributing the standard mp_eval per-source overhead.

6.4 Asymptotic comparison

For an arithmetization with σ instances of bit-ops over a common source column (e.g., SHA-256 with three instances each in σ_0 and σ_1, repeated across 48 message-schedule rounds), the construction reduces:

- committed column count by $\Theta(\sigma)$;
- proof-byte overhead from $\Theta(\sigma \log n)$ to $\Theta(\sigma)$ field elements;
- the equality-constraint count by exactly σ.

Because the witness commitment dominates honest-prover proof size in typical instantiations of the host PIOP, this cost reduction is proportional to the bit-op density of the arithmetization, not to its total constraint count.

6.5 Concrete numbers in the implementation

In the deployed Zinc+ SHA-256 UAIR (see the SHA UAIR specification), the use of the construction yields:

$$m \;=\; 6, \qquad |\mathcal{S}_{\mathrm{up}} \restriction_{\mathrm{binary}}| \;=\; 19,$$

i.e., the SHA UAIR declares six bit-op virtual columns over a 19-column binary-poly section of the trace. The bit-op virtual columns add exactly 6 field elements to the proof and 0 rounds.

7 Generalisation to $\mathbb{F}_2$-Linear Cell Operators

The proof of Theorem 3.4 uses only two facts about the operator σ:

- σ is $\mathbb{F}_2$-linear in the cell coordinate;
- σ acts trivially on the row coordinate (i.e., the same map at every row).

The coefficient-permutation structure of Theorem 3.1 was nowhere used. Hence the construction extends, verbatim, to any operator in the $\mathbb{F}_2$-linear envelope of bit-ops.

7.1 The $\mathbb{F}_2$-linear envelope

Definition 7.1 ($\mathbb{F}_2$-linear cell operator)**.** An $\mathbb{F}_2$-linear cell operator is any element of $\mathrm{End}_{\mathbb{F}_2}(R)$.

The space $\mathrm{End}_{\mathbb{F}_2}(R) \cong R \otimes_{\mathbb{F}_2} R^\vee$ has $\mathbb{F}_2$-dimension $32 \times 32 = 1024$, and is spanned by the elementary maps $E_{j,k} : X^k \mapsto X^j$. In particular it contains $\mathbb{F}_2$-linear combinations of bit-ops:

Example 7.2 (SHA-256 σ_0)**.** The SHA-256 small-sigma operator

$$\sigma_0(x) \;=\; \mathrm{Rot}_7(x) \oplus \mathrm{Rot}_{18}(x) \oplus \mathrm{Sh}^{\mathrm{R}}{}_3(x)$$

is the $\mathbb{F}_2$-linear combination $\mathrm{Rot}_7 + \mathrm{Rot}_{18} + \mathrm{Sh}^{\mathrm{R}}{}_3$ in $\mathrm{End}_{\mathbb{F}_2}(R)$. By linearity it lifts to a virtual column

$$\sigma_{0\,*}v \;=\; \mathrm{Rot}_{7\,*}\,v \;+\; \mathrm{Rot}_{18\,*}\,v \;+\; \mathrm{Sh}^{\mathrm{R}}{}_{3\,*}\,v$$

on cell-wise $\mathbb{F}_2$-arithmetic, and Theorem 3.4 applies.

7.2 Linearity of the construction

Lemma 7.3 (Linearity)**.** *Let $\sigma_1, \sigma_2 \in \mathrm{End}_{\mathbb{F}_2}(R)$ and let $a_1, a_2 \in \mathbb{F}_2$. Define $\sigma := a_1\sigma_1 + a_2\sigma_2$. Then*

$$\widetilde{\sigma_* v}(r) \;=\; a_1\widetilde{\sigma_{1\,*} v}(r) \;+\; a_2\widetilde{\sigma_{2\,*} v}(r),$$

and consequently

$$\psi_\alpha\big(\sigma(\overline{v}(r))\big) \;=\; a_1\,\psi_\alpha\big(\sigma_1(\overline{v}(r))\big) \;+\; a_2\,\psi_\alpha\big(\sigma_2(\overline{v}(r))\big).$$

Proof. Both equalities are immediate from $\mathbb{F}_2$-linearity of the MLE evaluation map and of ψ_α. □

Theorem 7.3 has a practical consequence: a UAIR that needs the virtual column $\sigma_{0\,*}v$ does *not* need to declare three separate bit-op specs and combine them downstream. It can instead declare a single *compound* spec (j, σ_0) once $\mathrm{End}_{\mathbb{F}_2}(R)$ is admitted as the domain of Theorem 3.1, with the verifier-side reconstruction

$$e \;=\; \psi_\alpha\big(\sigma_0(\overline{v_j}(r_0))\big) \;=\; \psi_\alpha\big((\mathrm{Rot}_7 + \mathrm{Rot}_{18} + \mathrm{Sh}^{\mathrm{R}}{}_3)(\overline{v_j}(r_0))\big),$$

still costing $O(32)$ $\mathbb{F}_q$-operations.

7.3 Beyond $\mathbb{F}_2$-linearity

The lemma fails for cell operators that are not $\mathbb{F}_2$-linear in the cell coordinate. A bit-mask $x \mapsto x \wedge \chi$ for a fixed mask $\chi \in R$ is $\mathbb{F}_2$-linear and therefore covered by Theorem 7.3. By contrast, a multiplicative operator such as $x \mapsto x \cdot y$ in R for a witness-dependent $y \in R$ is bilinear, not $\mathbb{F}_2$-linear in x alone, and does not fit the construction directly.

The construction in this paper covers the $\mathbb{F}_2$-linear case, which includes *all* bit-permuting and bit-masking operators of SHA-256 and SHA-3.

Conclusion

We have described a virtual-column construction that exploits the commutation between coefficient-position $\mathbb{F}_2$-linear endomorphisms of $R = \mathbb{F}_2[X]/(X^{32})$ and the multilinear-extension operator over the row coordinate. The construction adds *(i)* one MLE source to the constraint-aggregation sumcheck and to mp_eval; *(ii)* one field element of proof per spec; *(iii)* a closed-form verifier-side reconstruction at the final evaluation point. It introduces no fresh commitment, no fresh challenge, no extra protocol round, and no new soundness loss beyond the standard per-source mp_eval term. By Theorem 7.3, it covers arbitrary $\mathbb{F}_2$-linear functions of the 32 coefficient bits of a cell, which encompasses all bit-rotations, bit-shifts, bit-masks, and their XOR-combinations as they appear in SHA-256 and related primitives.